Shell scripts are just text files containing a sequence of commands that a shell (like bash) executes one after another. They're the simplest way to automate work on Linux/Termux — exactly the environment you're using for HiveNet nodes.

Basics

A shell script starts with a **shebang** that tells the system which interpreter to use:

```
Bash

#!/bin/bash
```

Then you write normal commands:

```
Bash

#!/bin/bash

echo "Starting HiveNode..."
date
echo "Battery check:"
termux-battery-status
```

Make it executable and run it:

```
Bash

chmod +x start-node.sh
./start-node.sh
```

Core Building Blocks

Variables

```
Bash

NODE_ID="node_$(date +%s)"
MESH_PORT=5000
echo "Node ID is $NODE_ID"
```

Conditionals

```bash
if [ -f /sdcard/hivenet/config.json ]; then
    echo "Config found"
else
    echo "No config — creating default"
fi
```

Loops

```bash
for i in 1 2 3; do
    echo "Attempt $i"
done

while true; do
    echo "Heartbeat $(date)"
    sleep 30
done
```

Functions

```bash
check_battery() {
    level=$(termux-battery-status | grep percentage | cut -d: -f2)
    echo "Battery: $level%"
}
```

Reading input / arguments

```bash
# $1, $2, etc. are command-line arguments
echo "First argument: $1"

# Interactive
read -p "Enter node name: " name
```

Useful Patterns for Your Setup (Termux + HiveNet)

Background services

```
Bash

#!/bin/bash
# start-mesh.sh
nohup python mesh_daemon.py > /sdcard/hivenet/logs/mesh.log 2>&1 &
echo $! > /sdcard/hivenet/mesh.pid
```

Safe shutdown

```
Bash

#!/bin/bash
if [ -f /sdcard/hivenet/mesh.pid ]; then
    kill $(cat /sdcard/hivenet/mesh.pid)
    rm /sdcard/hivenet/mesh.pid
fi
```

Logging with timestamps

```
Bash

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >>
/sdcard/hivenet/logs/node.log
}
```

Checking if a process is running

```
Bash

if pgrep -f "mesh_daemon.py" > /dev/null; then
    echo "Mesh is running"
else
    echo "Mesh is down — restarting"
    ./start-mesh.sh
fi
```

Simple menu

Simple menu

```bash
#!/bin/bash
echo "1) Start node"
echo "2) Stop node"
echo "3) Status"
read -p "Choice: " choice

case $choice in
    1) ./start-node.sh ;;
    2) ./stop-node.sh ;;
    3) ./status.sh ;;
    *) echo "Invalid" ;;
esac
```

Best Practices for Mobile / Off-grid

- Always use absolute paths or carefully manage `cd`.
- Log everything (battery, temperature, errors).
- Prefer `termux-` commands when available (`termux-battery-status`, `termux-wifi-connectioninfo`, etc.).
- Keep scripts small and single-purpose so they're easy to debug on a tablet.
- Use `set -e` near the top if you want the script to stop on any error.
- Quote variables: `"$VAR"` instead of `$VAR` (prevents word-splitting bugs).

Quick Start Example for You

```bash
```

```
#!/bin/bash
# hive-status.sh — quick health check

echo "=== HiveNode Status ==="
echo "Time: $(date)"
echo "Uptime: $(uptime -p 2>/dev/null || echo 'n/a')"
echo ""
echo "Battery:"
termux-battery-status 2>/dev/null || echo "termux-api not available"
echo ""
echo "Running processes:"
pgrep -af "python|mesh|ollama" || echo "No Hive processes found"
```

Would you like me to:

- Walk through a complete starter script for launching a HiveNode?
- Show how to make scripts that auto-start on Termux open?
- Cover more advanced topics (error handling, traps, background jobs, config files)?
- Or focus on something specific (networking checks, battery-aware throttling, container management)?

Just tell me the direction.